

Virtual Memory

COMS10015

Dr Tom Deakin

University of Bristol

Week 21

Virtual memory in practice (1)

```
#include <stdlib.h>
#include <stdio.h>

int main(void) {
    void *p = malloc((size_t)1024*1024*1024*1024*10); // 10 TB
    if (p == NULL)
        printf("Could not allocate\n");
    else
        printf("p=%p\n", p);
    return EXIT_SUCCESS;
}
```

Virtual memory in practice (2)

- ▶ What happens when I run that C program on my laptop (Mac OS)¹?

¹Note: on Linux, the C standard library (glibc) gives different output.

Virtual memory in practice (2)

- ▶ What happens when I run that C program on my laptop (Mac OS)¹?
- ▶ `$/a.out`
`p=0x7fbab9405800`
- ▶ I was given a valid (not NULL) pointer, but my laptop only has 8 GB of main memory.

¹Note: on Linux, the C standard library (glibc) gives different output.

Virtual memory

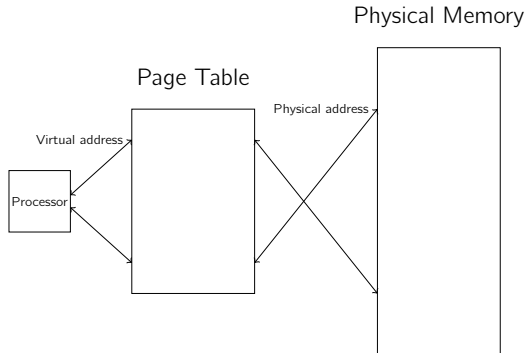
- ▶ In an ideal world, processors would have unlimited (main) memory.
- ▶ In reality, limited by the amount of physical memory.
- ▶ Kernel schedules multiple applications, so how much memory is available to each application?
- ▶ Use a *virtual memory* abstraction to give illusion of much larger memory than is available.
- ▶ Processor and kernel translate between the virtual and physical memory.
- ▶ Applications don't need to manage the physical memory, but kernel and architecture needs to instead.
- ▶ Kernel might move data through the memory hierarchy, e.g. between main memory and disk.

Pages

- ▶ Memory is split into *pages*.
- ▶ Pages normally contain 2^n bytes, where $n = [12, 21]$ (4 KiB–2 MiB).
- ▶ Architectures normally support a small number of page sizes.
- ▶ Say we allow a virtual memory size of 2^v , split into pages of 2^p bytes: 2^{v-p} pages.
- ▶ With 2^m bytes of physical memory, we can store 2^{m-p} of pages in physical memory.
- ▶ Recall: memory moved into cache in cache blocks.
- ▶ Memory moved in and out of physical memory in *pages*.

Virtual Addresses

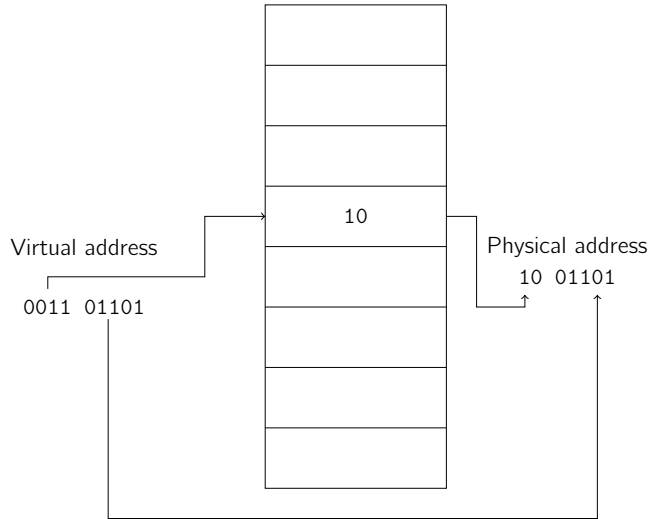
- ▶ Memory addresses used in the ISA are *virtual addresses*.
- ▶ Processor does an *address translation* to convert virtual address into *physical address*.
- ▶ Address translation occurs using a *Page Table*.



Page Table

- ▶ Page table is lookup table mapping virtual addresses to physical addresses.
- ▶ Typically stored in the main memory, at a location determined by the kernel.
- ▶ Also contains information on whether page in memory or disk, as well as location in the memory.
- ▶ Lookup table has 2^{v-p} entries (number of virtual pages) of size $(m - p)$ bits (the physical address).
- ▶ Most significant $v - p$ bits of virtual address used as index into page table.
- ▶ Page table gives $m - p$ of physical address.
- ▶ Remaining p bits of address used to index data *within* the page (page offset).

Example: Page Table



Page faults

- ▶ When processor accesses memory, uses the virtual address.
- ▶ Address translated to physical address via Page Table.
- ▶ If page is in memory, and memory access goes ahead.
- ▶ Otherwise, page is on disk and must be first moved to memory.
- ▶ This is called a *page fault*, and causes an interrupt/exception to transfer the page.
- ▶ Page faults also occur when a page is accessed for the first time.

Page replacement

- ▶ As virtual memory space is larger than physical memory, not all pages will fit in memory.
- ▶ If no space in physical memory during a page fault, must evict a page to disk.
- ▶ Seen a similar eviction with cache blocks in low levels of cache.
- ▶ Virtual memory systems behaves like a fully-associative cache as pages can go anywhere in physical memory.
- ▶ Latency to satisfy page fault very large, so kernel can afford to implement good but expensive (least recently used) replacement schemes.

Translation Lookaside Buffer

- ▶ Virtual memory forces an indirection.
- ▶ Two accesses **to memory** are required:
 1. Access page table to translate virtual address.
 2. Access data in physical memory.
- ▶ To optimise translation, use a cache!
- ▶ This cache is called the *Translation Lookaside Buffer* (TLB).
- ▶ Accessing the TLB is much faster than accessing memory (1 vs 100s clock cycles).
- ▶ Data caches rely on locality of data, so there is locality of the **addresses** of data.

Virtual memory protection

- ▶ Previously learnt about how the processor protects applications from accessing memory it shouldn't.
- ▶ With virtual memory, need to ensure the same.
- ▶ Permission information added to Page Table/TLB entries.
- ▶ Typically, three bits used to indicate if current process is able to read/write/execute at that address.
- ▶ Kernel maintains the Page Table so can only load in allowed translations when application is scheduled.

Further Reading

- ▶ Patterson and Hennessy, Computer Organization and Design, Chapter 5.7
- ▶ https://en.wikipedia.org/wiki/Virtual_memory
- ▶ [https://en.wikipedia.org/wiki/Page_\(computer_memory\)](https://en.wikipedia.org/wiki/Page_(computer_memory))
- ▶ https://en.wikipedia.org/wiki/Translation_lookaside_buffer
- ▶ https://en.wikipedia.org/wiki/Protection_ring
- ▶ <http://people.cs.bris.ac.uk/~dave/memh.pdf>
- ▶ <https://safari.ethz.ch/digitaltechnik/spring2018/lib/exe/fetch.php?media=onur-digitaldesign-2018-lecture25b-vm-afterlecture.pdf>